

Long-Form Reindexable RAG Architecture

Whitepaper

1. Problem Statement

A production RAG system must ingest heterogeneous documents, preserve source metadata, and answer questions with enough evidence for the LLM to cite where context came from. The system also needs a practical path for reindexing when the embedding model changes.

2. CocolIndex Incremental Indexing

CocolIndex is responsible for source tracking, transformation memoization, chunking, embedding, and writing vector rows into pgvector. A change in source input or transform logic should invalidate only affected rows instead of forcing every pipeline stage to rerun.

3. Embedding Model Migration

A same-dimension embedding model can be swapped with an in-place reindex. A different dimension model should use a shadow index or shadow column, then run an A/B comparison before traffic is switched to the new embedding representation.

4. Graphify Knowledge Graph

Graphify extracts a graph from code and documents. Graph nodes represent entities, classes, sections, and concepts. Edges carry relation labels and confidence tags such as EXTRACTED, INFERRED, and AMBIGUOUS. This helps downstream prompts weigh evidence more carefully.

5. Hybrid Retrieval

The query path first performs semantic vector search, then uses the retrieved chunk text and the natural language query to seed graph traversal. The expanded graph neighborhood adds entity-level evidence that a pure vector search may miss.

6. Drift Management

A dual-pipeline design has a measurable drift window because vector indexing and graph generation finish at different times. Production systems should track graph-known documents, vector-known documents, and alert when the symmetric difference exceeds an acceptable threshold.

7. Scaling Considerations

The local single-process API is useful for a POC but insufficient for high CCU. Production needs multiple API replicas, distributed cache, tuned vector indexes, and careful connection pool sizing so Postgres is not exhausted under load.

8. Lessons Learned

Reindexability must be designed before data is written. Generated graph artifacts must be excluded from the source corpus. Graph evidence is useful only when relation, confidence, source file, and source location are returned to the LLM.

9. Chunk Metadata

Each retrieved chunk should preserve a stable hash, section title, page number, and character span. These fields make duplicate detection, citation rendering, and debugging possible when the same paragraph appears in multiple places or after a document is regenerated.

10. Reranking and Deduplication

Vector search should overfetch candidates, remove exact duplicate chunks, and then rerank the remaining evidence with query-aware features. A cross-encoder reranker is the strongest option, while deterministic rules based on section titles are a practical baseline for this POC.

11. Evaluation Strategy

Retrieval quality should be evaluated with source recall, evidence rank, score margin, duplicate rate, and answer citation coverage. A good report records both the top vector score and the score of the first chunk that contains the expected evidence marker.

12. Deployment Checklist

Before production launch, the service should verify embedding dimensions, validate database indexes, warm caches for common queries, publish health metrics, and run rollback tests for schema changes involving shadow vector columns.